Deep Saliency Map Prediction for Human Eye Fixation Using FCN-ResNet50

Nasrul Huda
M.Sc. DSAI
University of Hamburg

Muhammad Bassam
M.Sc. IAS
University of Hamburg

Diksha Prasad
M.Sc. DSAI
University of Hamburg

***Abstract*—This report presents a PyTorch-based eye fixation prediction system that maps natural images to dense 224×224 saliency maps. The system uses a fully convolutional network with an ImageNet-pretrained ResNet-50 backbone, followed by Gaussian smoothing and a log center-bias prior. Training first uses a frozen backbone and then end-to-end fine-tuning, improving the best validation loss from 0.189 to 0.173. The fine-tuned run also reaches a peak mean validation ROC-AUC of 0.922. Qualitative analysis of validation fixation maps shows that single-object scenes provide concentrated targets, while multi-person scenes contain distributed attention patterns that are more difficult to model.**

I. Introduction

A. Objective

The objective of this project was to develop a deep learning system using PyTorch that predicts human eye fixation maps from natural images, producing a dense 224×224 saliency map for each input image that indicates where viewers are likely to look.

B. Approach Overview

We implemented a fully convolutional network (FCN) based on a pretrained ResNet-50 backbone with a single-channel output head. Raw network logits are post-processed with a fixed Gaussian smoothing kernel and a log center-bias prior to better match the spatial smoothness and central tendency of human fixations. The model was first trained with a frozen backbone and subsequently fine-tuned end-to-end. Training used binary cross-entropy loss on pixel-wise fixation density maps, with ImageNet normalization applied to input images.

II. System Architecture and Methodology

A. Foundational Work

Our implementation follows the FCN paradigm for dense prediction introduced by Long et al. [1], adapted from semantic segmentation to fixation and saliency map prediction as outlined in the course exercises [3]. The encoder uses a ResNet-50 backbone pretrained on ImageNet [2], accessed via PyTorch's `fcn_resnet50` implementation. Post-processing with a Gaussian kernel and center bias follows standard practice in computational saliency modeling to enforce spatial smoothness and account for the well-documented tendency of observers to fixate near image centers.

B. Network Design

Backbone and decoder. `FixationNet` uses torchvision's FCN-ResNet50 implementation with ImageNet-pretrained ResNet-50 backbone weights. The FCN head outputs a single channel of per-pixel logits at the input resolution (224×224), treating each pixel as an independent Bernoulli target rather than a class label in a softmax sense.

Post-processing modules. Two non-trainable components refine the raw output:

1) **Gaussian smoothing:** A separable 25×25 Gaussian kernel ($\sigma = 11.2$) is applied via `conv2d` with "same" padding, producing spatially smooth saliency maps consistent with aggregated eye-tracking data.
2) **Center bias:** A dataset-provided center-bias density map, `center_bias_density.npy`, is stored in log-space and added to the smoothed logits, encoding the prior that fixations tend to fall near the image center.

The forward pass is summarized as

$$\text{FCN logits} \rightarrow \text{Gaussian smoothing} \rightarrow \text{log center bias} \rightarrow \text{output logits},$$

with a sigmoid applied at inference.

C. Modifications and Rationale

We experimented with two training regimes, summarized in Table I.

Regime	Trainable layers	Best loss	Best/total
Frozen	FCN head	0.189	16 / 88
Fine-tuned	Full network	**0.173**	24 / 38

TABLE I
Training regimes compared during development.

Initially, the ResNet-50 backbone was frozen to stabilize early training and reduce overfitting on the limited dataset. After observing a validation plateau around 0.19, we unfroze the full network and continued training end-to-end. Fine-tuning improved the best validation loss by approximately 8.3% (0.189 to 0.173). The peak validation ROC-AUC also increased from 0.895 in the frozen run to **0.922** in the fine-tuned run, indicating that task-specific adaptation of low-level and mid-level features helps capture image-dependent saliency beyond what the generic center bias and frozen features can provide.

No additional architectural changes, such as attention modules or skip-connection modifications, were introduced. This kept the model aligned with the course template while focusing optimization effort on the training strategy.

III. Training

A. Dataset and Preprocessing

Splits. We used the course-provided fixation dataset with the official split files:

- **Training:** 3,006 image–fixation pairs.
- **Validation:** 1,128 image–fixation pairs.
- **Test:** 1,032 images, listed in `test_images.txt`, used for final prediction submission.

Input format. Images are RGB at 224×224 pixels. Ground-truth fixation maps are single-channel grayscale density maps at 224×224, with pixel values in $[0, 255]$ converted to $[0, 1]$ via `ToTensor()`.

Augmentation. No data augmentation was applied in the final pipeline. Images were only converted to tensors and normalized. This choice keeps preprocessing simple and ensures validation metrics remain directly comparable across epochs; future work could add horizontal flips or color jitter to improve generalization.

Normalization. Input images were normalized with ImageNet statistics: mean = $[0.485, 0.456, 0.406]$ and standard deviation = $[0.229, 0.224, 0.225]$. Fixation maps received only `ToTensor()` scaling to $[0, 1]$.

B. Loss Function

We trained with **binary cross-entropy with logits**, implemented as `F.binary_cross_entropy_with_logits`. This loss compares the model's per-pixel logits to the ground-truth fixation density after scaling to $[0, 1]$. It is well suited for dense fixation prediction because each pixel is treated as an independent probability target, allowing the network to learn both high-saliency regions and background without requiring a normalized probability distribution over the image.

C. Hyperparameters

The final model was selected using the lowest validation loss observed during fine-tuning. The training configuration is shown in Table II.

Hyperparameter	Value
Optimizer	Adam
Learning rate	1×10^{-4}
Training batch size	16
Validation batch size	64
Dropout	None
Epochs trained (fine-tuning)	38, best at epoch 24
Total planned epochs	100, with final selection by best validation loss

TABLE II
FINAL FINE-TUNING HYPERPARAMETERS.

Checkpoints were saved whenever validation loss improved, allowing training to resume from the best saved model state.

IV. Results

A. Quantitative Results

The best validation loss achieved during fine-tuning was **0.1730** at epoch 24. Mean validation ROC-AUC was computed per image by applying a sigmoid to the model logits and binarizing ground-truth fixation pixels at 0.5. The ROC-AUC at the best-loss epoch was 0.918, and the peak ROC-AUC over the fine-tuning run was **0.922** at epoch 38.

Training and validation loss curves over 38 fine-tuning epochs are shown in Fig. 1. Training loss decreased steadily from 0.319 to 0.125, while validation loss dropped sharply in the first few epochs and then plateaued around 0.173–0.177, suggesting mild overfitting in later epochs but stable generalization after the best checkpoint.

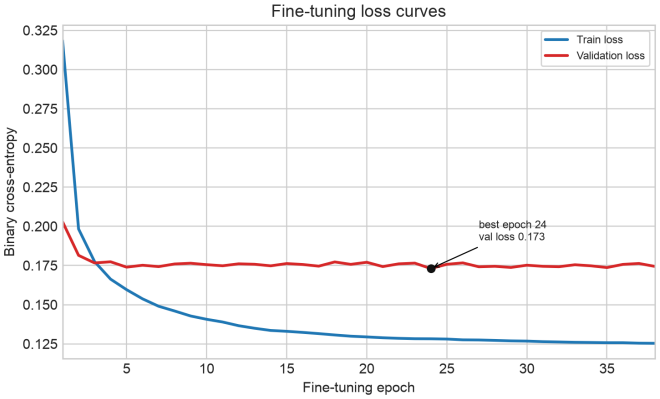

Fig. 1. Training and validation binary cross-entropy loss over 38 fine-tuning epochs.

Fig. 2 compares the frozen-backbone and fine-tuned training histories. The frozen model improves quickly but remains near a validation-loss floor of 0.19, whereas the fine-tuned model consistently operates at lower validation loss and higher ROC-AUC.

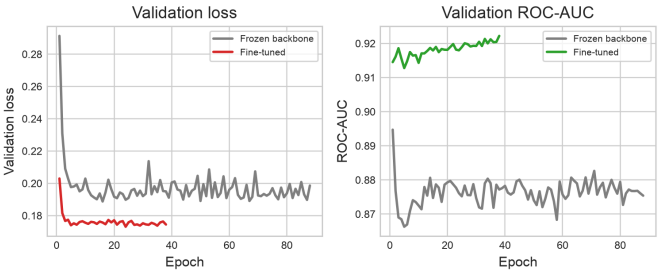

Fig. 2. Validation loss and ROC-AUC for the frozen-backbone and fine-tuned training regimes.

Metric	Frozen backbone	Fine-tuned
Val loss	0.189	**0.173**
Peak val ROC-AUC	0.895	**0.922**
Best-loss epoch	16	24

TABLE III
VALIDATION PERFORMANCE FOR THE FROZEN AND FINE-TUNED MODELS.

B. Qualitative Analysis

The available validation images and fixation maps illustrate two important target structures for this task: compact, single-object fixation maps and distributed, multi-object fixation maps.

1) Concentrated case: `image-4026`*:* Fig. 3 shows a validation image with a single person in an open landscape. The ground-truth fixation density is compact and centered on the person, making this type of example well matched to the model's Gaussian smoothing and center-bias assumptions.

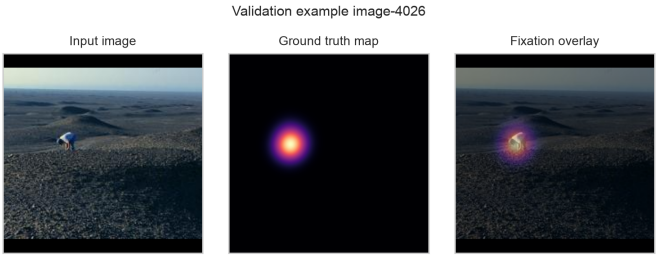

Fig. 3. Validation example with a compact fixation target. The fixation map is concentrated around the single visible person.

2) Distributed case: `image-3829`*:* Fig. 4 shows a multi-person golf-course scene. The ground-truth fixation map is spread across people, gestures, the golf bag, and background regions. Such examples are more challenging because the target is multi-modal and depends on semantic relationships between objects rather than only local contrast or centrality.

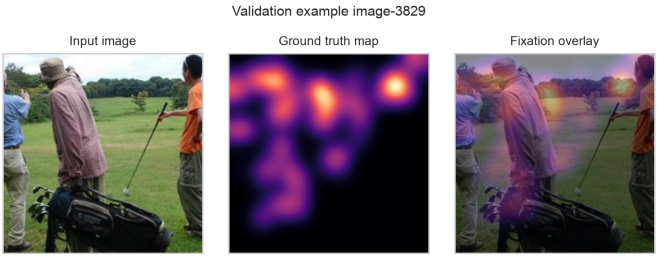

Fig. 4. Validation example with distributed fixation targets. The target map contains several separated high-attention regions across people and contextual objects.

These examples explain the main qualitative limitation observed during development: the model is better suited to scenes with a single dominant salient object than to scenes where human attention is distributed across several semantic regions.

V. CONCLUSION

We built an eye fixation prediction system using an FCN-ResNet50 architecture with Gaussian smoothing and center-bias post-processing, trained on 3,006 images and validated on 1,128. Starting from a frozen ImageNet backbone with best validation loss 0.189, end-to-end fine-tuning yielded a best validation loss of **0.173** and a peak validation ROC-AUC of **0.922**. Qualitative inspection of validation targets confirms that scenes with clear, isolated salient objects are structurally easier than complex multi-person images with distributed fixation regions.

Key lessons include the value of fine-tuning over frozen features for this task and the importance of inspecting failure modes. Border artifacts and missed secondary fixations point to limitations of pixel-wise BCE without explicit handling of sparsity or multi-modal attention.

Future work should explore KL-divergence or correlation-based losses better suited to sparse, probabilistic fixation maps. Further improvements could also add data augmentation or an attention mechanism to improve performance on multi-object scenes.

REFERENCES

[1] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
[3] Course materials, Computer Vision 2, Exercise 5 — FCN for Eye Fixation Prediction, University of Hamburg, 2026.